TP — Algorithmie Avancée III

ESGI — M1 — 2026-2027

Consignes

- Binôme. Python 3, bibliothèque standard uniquement.
- Pas de threads ni d'`asyncio` : tous les algorithmes répartis s'exécutent dans le simulateur de l'exercice 1.
- Chaque expérience utilise une graine aléatoire fixée, indiquée dans `results/`.
- Le bonus n'est pas obligatoire.
- Rendu : archive `NOM1_Prenom1_NOM2_Prenom2.zip` (`README.md`, `src/`, `tests/`, `results/`, sans `__pycache__`) à envoyer à **jherard1@myges.fr**, objet `[ESGI M1 Algo III] NOM1 NOM2`.

Partie 1 — Modèle réparti et algorithmes à vague

Exercice 1 — Simulateur à événements discrets

Le simulateur remplace le réseau. Un processus ne connaît que son identifiant et la liste de ses voisins. Il réagit à trois événements : démarrage, réception d'un message, fin d'une minuterie.

1. Écrivez une classe `Processus(pid, voisins)` avec les méthodes `envoyer(dest, contenu)`, `decider(valeur)`, `armer_minuterie(delai, etiquette)` et les réactions `au_demarrage()`, `a_la_reception(source, contenu)`, `a_la_minuterie(etiquette)`, vides par défaut.

2. Écrivez une classe `Reseau(processus, graine, delai_max)` :

- file d'événements triée par instant (`heapq`) ;
- délai de chaque message tiré uniformément dans $[1, \text{delai_max}]$ avec `random.Random(graine)` ;
- `executer(initiateurs)` traite les événements jusqu'à épuisement ;
- attribut `messages` : nombre total de messages envoyés ;
- envoi vers un processus qui n'est pas voisin : `ValueError`.

3. Les canaux doivent être FIFO. Des délais tirés indépendamment ne le garantissent pas. Corrigez le calcul de l'instant d'arrivée, puis écrivez un test qui envoie 20 messages numérotés sur un même canal.

4. Ajoutez une horloge de Lamport : +1 à chaque envoi et à chaque décision, `max(locale, estampille) + 1` à chaque réception. Vérifiez que l'horloge croît strictement le long d'une chaîne causale. Deux événements d'horloges différentes sont-ils forcément liés causalement ?

5. Ajoutez un ensemble `pannes` : un processus en panne ne démarre pas et perd les messages qui lui arrivent.

6. Écrivez les générateurs `anneau_oriente(n)`, `anneau_bidirectionnel(n)`, `arbre_aleatoire(n, graine)` et `graphe_connexe_aleatoire(n, m, graine)`.

Exercice 2 — Vague sur un anneau

1. Anneau orienté, initiateur 0. L'initiateur envoie un jeton à son successeur, chaque autre processus le relaie, l'initiateur décide à sa réception. Implémentez `VagueAnneau`.

2. Vérifiez les trois propriétés d'une vague (terminaison, décision, dépendance). Quels événements précèdent causalement la décision ?

3. Mesurez le nombre de messages pour n = 5, 20 et 100.

Exercice 3 — Vague sur un arbre

Le réseau est un arbre et tous les processus sont initiateurs. Un processus envoie un message au voisin `q` dès qu'il a reçu un message de tous ses autres voisins. Il décide quand il a reçu un message de tous ses voisins.

1. Implémentez `VagueArbre`.

2. Mesurez le nombre de messages sur des arbres aléatoires de 5, 20 et 100 sommets, pour plusieurs graines. Comparez à la valeur annoncée par le cours, puis à $2(n-1)$, coût de l'écho sur un arbre. D'où vient l'écart ?

3. Combien de processus décident ? Montrez qu'ils sont voisins.

4. Que se passe-t-il si le réseau contient un cycle ?

Exercice 4 — Vague d'écho

1. Implémentez `Echo` (initiateur unique, graphe connexe). À sa première réception, un non-initiateur mémorise l'émetteur comme père et envoie à tous ses autres voisins. Quand il a reçu un message de tous ses voisins, il envoie au père. L'initiateur décide quand il a reçu un message de tous ses voisins.

2. Écrivez `est_arbre_couvrant(g, racine, peres)` et vérifiez que la relation père forme un arbre couvrant.

3. Mesurez le nombre de messages pour $(n, |E|) = (10, 15)$, $(50, 120)$ et $(100, 300)$. Comparez à $2|E|$.

4. Changez la graine. L'arbre couvrant change-t-il ? Le nombre de messages ?

5. Relevez l'horloge de Lamport de la décision et comparez-la au nombre de messages. Pourquoi est-elle plus petite ?

Partie 2 — PIF et élection

Exercice 5 — Propagation d'information avec feedback

Chaque processus `i` possède une valeur v_i. La racine veut la somme et le maximum des valeurs, et savoir que tous les processus ont reçu son message.

1. Implémentez `PIF`. Propagation : message `msg`. Feedback : `ack` portant la somme et le maximum du sous-arbre. Un non-racine envoie son `ack` au parent après avoir reçu un message (`msg` ou `ack`) de chacun de ses voisins.

2. Sur 10 graphes aléatoires, vérifiez que la racine obtient la somme et le maximum exacts et que le nombre de messages vaut $2|E|$.

3. Une version naïve du PIF, proche de celle du cours mais avec une autre condition de boucle, s'écrit ainsi :

```
parent, acks_needed = None, len(neighbors)
...
while acked < (acks_needed - (0 if is_root else 1)):
    (kind, src, m) = inbox.get()
    if kind == "msg" and parent is None and not is_root:
        parent = src
        ...
    acked += 1
```

Combien de messages un non-racine attend-il, message du parent compris ? Ajoutez un paramètre `erreur_cours=True` qui reproduit ce comportement et exhibez un contre-exemple. Que fait une feuille avec ce code ?

4. Pourquoi le PIF permet-il de détecter la terminaison de la diffusion, et pas le PI seul ?

Exercice 6 — Chang-Roberts

1. Implémentez `ChangRoberts` sur un anneau orienté, puis l'annonce du leader. Comptez à part les messages d'élection.

2. Mesurez les messages d'élection pour n = 10, 50 et 100 : identifiants croissants dans le sens des messages, décroissants, puis moyenne sur 200 permutations aléatoires.

3. Retrouvez par le calcul les valeurs des deux premiers cas. Comparez la moyenne à $n \ln n$ et à nH_n, avec $H_n = \sum_{k=1}^{n} 1/k$.

4. Pourquoi aucun algorithme déterministe ne peut-il élire un leader sur un anneau anonyme ?

Exercice 7 — Bully

Réseau complet, l'identifiant d'un processus est son numéro. Le processus qui détecte la panne du leader lance une élection.

1. Implémentez `Bully` avec des minuteries. `ELECTION` vers les identifiants supérieurs, attente d'un `OK` pendant `T`. Sans réponse : diffusion de `COORDINATOR`. Avec réponse : attente d'un `COORDINATOR`, sinon nouvelle élection.

2. Quelle valeur de `T` garantit qu'un processus vivant a le temps de répondre ? Quelle hypothèse sur le réseau cela suppose-t-il ?

3. Le leader n - 1 est en panne. Mesurez les messages quand la panne est détectée par le processus 0, puis par le processus n - 2, pour n = 5, 10 et 20. Retrouvez les formules.

4. Recommencez avec deux pannes (n - 1 et n - 2), détection par le processus 0.

Exercice 8 — Élection par écho avec extinction

Chaque processus lance une vague d'écho portant son identifiant. Un processus rejoint une vague d'identifiant supérieur à sa vague courante et ignore les vagues inférieures. Seule la vague maximale revient à son initiateur, qui devient leader.

1. Implémentez `EchoExtinction`.

2. Sur un graphe complet de 10, 20 et 40 processus, tous initiateurs, mesurez les messages et comparez à $2|E|$.

3. Le cours annonce $O(n \cdot |E|)$ messages au pire. Construisez le cas suivant : ligne $0 - 1 - \cdots - (n-1)$, tous initiateurs à l'instant 0, délai 1 vers la gauche et 1000 vers la droite. Ajoutez au simulateur un paramètre `delai(src, dst)`. Mesurez pour n = 10, 20, 40 et 80. Quel est l'ordre de grandeur ? La borne du cours est-elle atteinte ? Que vaut le coût d'une vague isolée ?

Bonus — Hirschberg-Sinclair

Anneau bidirectionnel. En phase `k`, chaque candidat envoie une sonde à distance 2^k dans les deux sens. Une sonde est absorbée par un identifiant supérieur, sinon elle revient sous forme de réponse. Un candidat qui reçoit ses deux réponses passe à la phase `k + 1`. Il est élu quand sa propre sonde lui revient.

1. Implémentez `HirschbergSinclair` et mesurez les messages pour n = 16, 64 et 256 (permutation aléatoire des identifiants). Comparez à $8n \log_2 n$ et à Chang-Roberts, en moyenne et dans le pire cas.

Partie 3 — Classes de complexité et réductions

Exercice 9 — Espace logarithmique

L'entrée est en lecture seule. Seul l'espace de travail compte.

1. Écrivez `est_palindrome_logspace(mot)` qui n'utilise que deux indices. Comparez à `mot == mot[::-1]` sur des mots aléatoires.

2. Écrivez `bits_de_travail(n)`, nombre de bits nécessaires pour les deux indices. Donnez sa valeur pour n = 10, 1000 et 10^6.

3. Pourquoi la version `mot[::-1]` n'est-elle pas en espace logarithmique ?

Exercice 10 — STCON

Graphe orienté à n sommets donné par ses listes de successeurs. Existe-t-il un chemin de `s` à `t` ?

1. Écrivez `stcon_bfs(n, succ, s, t)`. Complexité en temps et en espace ?

2. Écrivez `stcon_devine(n, succ, s, t, choix)`, qui exécute une branche du calcul non déterministe : `choix(courant, etape)` désigne le successeur choisi. Quelles variables sont conservées ? Combien de bits ?

3. Écrivez `stcon_toutes_branches(n, succ, s, t)`, qui essaie toutes les suites de choix. Pourquoi n étapes suffisent-elles ? Combien de branches ?

4. Écrivez `savitch(n, succ, u, v, k)` : existe-t-il un chemin de `u` à `v` de longueur au plus 2^k ? Passez par un sommet milieu `w`. Donnez la profondeur de récursion, l'espace et le temps.

5. Vérifiez l'accord des trois méthodes sur 100 graphes aléatoires de 7 sommets. Mesurez ensuite les temps pour n = 6, 7 et 8 sur le graphe où `t = n - 1` n'a aucun arc entrant et où les autres sommets sont tous reliés. Reliez les résultats à **NL** $\subseteq$ **P** et au théorème de Savitch.

Exercice 11 — Vérifier et résoudre SAT

Une formule CNF est une liste de clauses. Une clause est une liste d'entiers : `x` pour la variable `x`, `-x` pour sa négation.

1. Écrivez `verifier_sat(formule, affectation)`. Complexité ?

2. Écrivez `sat_exhaustif(formule, nb_vars)`, qui renvoie une affectation satisfaisante ou `None`, et le nombre d'affectations essayées.

3. Mesurez la vérification d'une affectation pour 20 variables et 85 clauses. Mesurez la résolution exhaustive d'une formule insatisfiable à 12, 16 et 20 variables. Estimez le temps pour 40 variables.

4. En quoi cette expérience illustre-t-elle la définition de NP par certificat ?

Exercice 12 — Réduction de CLIQUE à VERTEX COVER

CLIQUE : le graphe G contient-il une clique de taille k ? VERTEX COVER : G possède-t-il un ensemble de k sommets qui touche toutes les arêtes ?

1. Écrivez `est_couverture(aretes, sommets)`. Pourquoi VERTEX COVER est-il dans NP ?

2. Montrez que C est une clique de G si et seulement si $V \setminus C$ est une couverture du complémentaire $\overline{G}$.

3. Écrivez `reduction_clique_vc(n, aretes, k)`, qui renvoie $(n, \overline{G}, n-k)$. Complexité ?

4. Écrivez les résolutions exhaustives `a_une_clique` et `a_une_couverture`. Vérifiez l'équivalence $(G,k) \in \text{CLIQUE} \iff f(G,k) \in \text{VERTEX COVER}$ sur 300 instances aléatoires ($n \leq 7$).

5. CLIQUE est NP-complet. Que peut-on conclure pour VERTEX COVER ? Pourquoi la réduction doit-elle aller de CLIQUE vers VERTEX COVER et pas l'inverse ?

Exercice 13 — 2-SAT et 3-SAT

1. Pour une formule 2-CNF, la clause $(a \vee b)$ donne les arcs $\neg a \to b$ et $\neg b \to a$. Écrivez la construction du graphe d'implication.

2. Calculez les composantes fortement connexes (Kosaraju ou Tarjan, version itérative). La formule est satisfiable si et seulement si aucune variable `x` n'est dans la même composante que $\neg x$. Déduisez une affectation de l'ordre topologique des composantes. Écrivez `deux_sat(formule, nb_vars)`.

3. Vérifiez l'accord avec `sat_exhaustif` sur 300 formules aléatoires et vérifiez chaque affectation renvoyée.

4. Mesurez `deux_sat` pour 100 000 variables avec 50 000 puis 200 000 clauses. Comparez aux temps de l'exercice 11.

5. 2-SAT est NL-complet, 3-SAT est NP-complet. Pourquoi le graphe d'implication ne s'applique-t-il plus avec des clauses de trois littéraux ?